AMANE — Refonte V3

Faire d'AMANE une enveloppe d'exécution souveraine pour SaaS externes

Journal de refonte, décisions d'architecture et tri du dépôt

Version	1.4
Date	2026-10-28
Branche	main3
Statut	Étape 2 en cours — 2a terminée (laboratoire HA), 2b à venir

Sommaire

Fiche du document 3

Historique des versions . . . 3

1. Objet de la refonte 4

1.1 Ce qu'est une « enveloppe d'exécution » . . . 4

1.2 La promesse à tenir . . . 4

2. État actuel du dépôt (V2, point de départ) 5

2.1 Point de départ figé . . . 5

2.2 Cartographie des composants . . . 5

2.3 Trois constats qui orientent le tri . . . 5

3. Décisions d'architecture 7

ADR-001 — Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go . . . 7

ADR-002 — Formats d'exécutables acceptés . . . 7

ADR-003 — Zero-knowledge strict . . . 8

ADR-004 — Modalités de la haute disponibilité PME . . . 8

4. Tri du dépôt 10

4.1 À garder intact . . . 10

4.2 À garder mais adapter . . . 10

4.3 À supprimer . . . 11

5. Plan de travail par étapes 12

6. Journal des étapes 13

Étape 0 — Préparation (2026-10-28) . . . 13

Étape 1 — Nettoyage (2026-10-28) . . . 13

Étape 1 — Compléments (2026-10-28) . . . 14

Étape 2a — Image de l'orchestrateur et laboratoire HA (2026-10-28) . . . 14

7. Points ouverts 19

Annexe — Mettre à jour ce document 20

Fiche du document

Ce document est le **journal de bord de la refonte V3** d'AMANE. Il récapitule l'état du dépôt au départ, le tri de chaque composant (garder intact, adapter, supprimer), les décisions d'architecture prises et leur justification, puis l'avancement étape par étape. Il sert de référence aux collaborateurs pour comprendre **ce qui a été fait, pourquoi, et dans quel ordre**.

Il est mis à jour à chaque étape de la refonte. La source fait foi (`docs/refonte-v3/AMANE-Refonte-V3.md`) ; le PDF est régénéré à partir d'elle (voir l'annexe « Mettre à jour ce document »).

Historique des versions

Version	Date	Branche / commit	Changements
1.0	2026-10-28	main3 / 6da5022	Création : état V2, tri initial, décisions ADR-001 à ADR-003, plan par étapes
1.1	2026-10-28	main3 / dc06197	Étape 1 (nettoyage) terminée ; tri corrigé (§ 4.3) ; points à vérifier ajoutés (§ 7)
1.2	2026-10-28	main3	Compléments à l'étape 1 : compilation Rust confirmée, test de réplication corrigé, test V1 `mock_server.rs` retiré ; réplication synchrone ajoutée à ADR-001 et à l'étape 2
1.3	2026-10-28	main3	ADR-004 (modalités de la HA PME, 7 décisions) ; `quorum.rs` réorienté plutôt que supprimé ; étape 2 découpée en 2a à 2d
1.4	2026-10-28	main3	Sous-étape 2a : image de l'orchestrateur, laboratoire HA trois machines, 10 scénarios de chaos réussis, 3 défauts de l'orchestrateur corrigés

1. Objet de la refonte

1.1 Ce qu'est une « enveloppe d'exécution »

Jusqu'en V2, AMANE embarquait sa propre application métier (gestion de stock) au-dessus de son socle de souveraineté. La V3 inverse la logique : **AMANE devient l'enveloppe dans laquelle s'exécute un SaaS édité par un tiers**.

- Un éditeur externe soumet l'**exécutable** de son SaaS à AMANE (en priorité une image Docker).
- AMANE le déploie sur le cluster de la PME et en assure l'**orchestration** : haute disponibilité, réplication, bascule, réseau, sauvegarde.
- Le SaaS s'exécute **sur la base de l'architecture AMANE** : ses données restent chez la PME et ne sortent que chiffrées.
- L'**interface utilisateur et la couche métier du SaaS ne changent pas**. L'utilisateur final retrouve exactement l'application qu'il connaît ; l'éditeur externe n'a pas à réécrire son code ni à intégrer un SDK.

1.2 La promesse à tenir

> **Personne d'autre que la PME ne voit de données en clair. La DEK et toutes les clés de chiffrement restent dans le cluster PME. Ni l'éditeur AMANE, ni le relais, ni l'éditeur du SaaS externe n'ont accès au clair.**

Cette promesse est le cœur d'AMANE ; toute décision de la refonte est évaluée à son aune.

2. État actuel du dépôt (V2, point de départ)

2.1 Point de départ figé

- Branche de travail créée : `main3`, à partir de `main2` (commit `e0f6d09`), poussée sur `origin/main3`.
- Le travail V2 non commité (76 fichiers) a été figé dans le commit `6da5022` « chore: état V2 avant nettoyage ». C'est le point de retour de référence avant toute suppression.

2.2 Cartographie des composants

Composant	Chemin	Rôle en V2
SaaS éditeur (Django)	`config/`, `tenants/`, `licenses/`, `devices/`, `dashboard/`, `api/`, `templates/`	Comptes tenants, licences, parc installé, génération des installeurs PME
Console web (React)	`frontend/`	Console éditeur et espace tenant, consomme l'API DRF
Orchestrateur cluster (Go)	`orchestrator-go/`	etcd v3, Patroni, réplication CRDT, mesh WireGuard, serveur gRPC mTLS
Socle crypto et journal (Rust)	`spike/crates/ss-crypto`, `ss-journal`, `ss-consensus`	AK/DEK, CRL, Shamir, journal CBOR chiffré, époque et fencing, garde de quorum
SDK client métier (Rust)	`core-rust/ss-client/`	Invariants stock, mode hors-ligne, appairage, licence, client gRPC
Nœud PME (Rust)	`spike/node/`	CLI, supervision PostgreSQL, annonce au relais, **application stock et utilisateurs intégrée**
Relais zero-knowledge (Rust)	`spike/relay/`, `relay-deploy/`	Stockage de blobs opaques et topologie du cluster
Contrat gRPC	`proto/amane/framework/v1/framework.proto`	Interface unique B ↔ C (Ping, Enroll, Write, Read, NotifyRevocation, PushDelta)
Déploiement PME	`pme-deploy/`, `templates/tenants/*.tmpl`	PostgreSQL primaire/standby + nœud + pgAdmin + watchtower
CI et mesures	`.github/workflows/`, `.ci/`, `scripts/`, `tests/`, `docs/bench/`	Build, couverture, chaos, contrats, benchmarks, mesures de bascule

2.3 Trois constats qui orientent le tri

Constat 1 — Deux mécanismes de haute disponibilité coexistent. L'orchestrateur Go (etcd + Patroni) n'est utilisé que dans le `docker-compose.yml` de développement et dans la CI. La pile réellement déployée chez la PME n'en contient pas : elle repose sur une réplication streaming PostgreSQL nue et sur le nœud Rust, qui décide seul de la promotion d'un standby (`spike/crates/ss-consensus/src/quorum.rs`, élection déterministe par LSN). Cette élection écrite à la main est en tension avec la règle n° 2 du projet (« aucun algorithme de consensus écrit à la main »).

Constat 2 — Le nœud PME embarque sa propre application métier. Le module `spike/node/src/stock/` et les pages HTML associées (articles, utilisateurs, tableau de bord) constituent une application métier complète. C'est précisément le rôle que tiendra désormais le SaaS externe.

Constat 3 — La console éditeur existe en double. Une version rendue côté serveur par Django (`templates/*.html`, `views_web.py`, `views.py`) et une version React (`frontend/`) couvrent les mêmes

écrans.

Autres observations : le nœud déclare la dépendance `ss-client` dans le workspace mais ne l'utilise pas ; un fichier proto V1 (`proto/framework.proto`) déclare le même paquet que le contrat qui fait autorité ; plusieurs scripts et notes isolés traînent à la racine.

3. Décisions d'architecture

Chaque décision est consignée sous forme d'ADR (Architecture Decision Record) : contexte, décision, conséquences. Une décision n'est jamais effacée : si elle est remplacée, elle passe au statut « Remplacée par ADR-xxx ».

ADR	Titre	Statut	Date
ADR-001	Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go	Acceptée	2026-10-28
ADR-002	Formats d'exécutables acceptés : Docker d'abord, binaires et .exe ensuite	Acceptée	2026-10-28
ADR-003	Zero-knowledge strict : aucun clair hors du cluster PME, clés confinées	Acceptée	2026-10-28
ADR-004	Modalités de la HA PME : témoin etcd, synchrone strict dès 3 nœuds, garde de quorum en lecture	Acceptée	2026-10-28

ADR-001 — Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go

Contexte. Deux mécanismes de HA coexistent (constat 1). La garde de quorum Rust (`quorum.rs`) réalise une élection de primaire écrite à la main, ce que la règle n° 2 interdit. L'orchestrateur Go, lui, s'appuie sur etcd (Raft) et Patroni, est déjà couvert par les tests d'intégration et par les mesures de bascule de la CI (switchover ~2,2 s, failover ~3,2 s).

Décision. La pile PME adopte **etcd + Patroni, piloté par l'orchestrateur Go**. C'est le seul mécanisme de bascule et de fencing en production.

Conséquences.

- Les compose PME (`pme-deploy/`, `templates/tenants/docker-compose.yml.tmpl`) intègrent etcd, Patroni (spilo) et l'orchestrateur Go, à l'image du compose de développement.
- Le nœud Rust ne promeut plus PostgreSQL lui-même (`run_supervision_loop`, sonde des pairs, `cmd/failover.rs`, `promote_standby` retirés). `quorum.rs` n'est pas supprimé mais **réorienté** en garde de quorum en lecture (voir ADR-004).
- La règle « ≥ 3 nœuds → failover automatique par quorum » est portée par etcd/Patroni. Le cas des clusters à 2 serveurs PostgreSQL est tranché par ADR-004 (témoin etcd).
- L'époque et le fencing (`epoch.rs`, `fencing.rs`) restent dans le socle ; leur source et leur contrôle continu sont fixés par ADR-004.
- **Réplication synchrone effective.** La pile V2 positionne `synchronous_commit=remote_write` sans définir `synchronous_standby_names` : PostgreSQL valide alors chaque transaction sur la seule écriture locale, la réplication est en réalité asynchrone. Cela contredit la règle « synchrone pour les invariants forts, blocage si aucun passif n'est joignable ». L'étape 2 la rend effective via Patroni (`synchronous_mode` et `synchronous_mode_strict`), qui bloque les écritures plutôt que de dégrader silencieusement en asynchrone (décision du 2026-10-28).

ADR-002 — Formats d'exécutables acceptés

Contexte. Un SaaS externe peut être livré sous plusieurs formes : image Docker, binaire Linux, exécutable Windows (.exe). Supporter tous les formats d'emblée disperserait l'effort.

Décision. AMANE accueillera **plusieurs types d'exécutables**. L'effort porte d'abord **exclusivement sur les images Docker**. Les binaires et les .exe seront développés ensuite.

Conséquences.

- Le format de description d'une application soumise (le « manifeste ») porte dès le départ un champ **type d'artefact** (`docker`, puis `binaire`, `exe`), afin que l'ajout ultérieur des autres formats n'oblige pas à casser le modèle de données ni le contrat.
- Les images sont référencées par **digest** (`sha256:…`) et non par tag, pour que la PME exécute exactement ce qui a été soumis.
- Piste pour la suite (non tranchée) : envelopper un binaire nu dans une image de base fournie par AMANE ; un .exe Windows demandera des conteneurs Windows ou une machine virtuelle.

ADR-003 — Zero-knowledge strict

Contexte. La promesse d'AMANE est que seule la PME voit ses données en clair. En V3 un acteur nouveau entre dans le périmètre : l'éditeur du SaaS externe, dont le code s'exécute **à l'intérieur** du cluster PME et manipule donc du clair.

Décision. Le zero-knowledge est maintenu sans compromis : **personne d'autre que la PME ne voit de clair**, et **la DEK comme toutes les clés de chiffrement restent dans le cluster PME**.

Conséquences.

- **Confinement réseau** : l'application externe s'exécute sur un réseau interne sans sortie vers Internet par défaut. Seuls sont exposés son interface utilisateur (via le reverse proxy du nœud) et sa base de données gérée par AMANE. Elle ne peut pas renvoyer de données en clair à son éditeur.
- **Chiffrement au repos** : les données de l'application (base, volumes) sont chiffrées sous des clés dérivées de la hiérarchie AMANE ; la DEK ne quitte jamais le cluster.
- **Sortie uniquement chiffrée** : sauvegardes, réplication inter-sites et tout transit par le relais sont chiffrés côté PME avant envoi. Le relais ne voit que des blobs opaques.
- **Télémétrie** : le SaaS éditeur AMANE ne reçoit que des métadonnées (état du cluster, licence, santé), jamais de contenu métier.
- **Aucun composant hors PME** (Django, relais, éditeur externe, agent IA) ne touche AK/DEK. Les opérations sur les clés restent humaines.

ADR-004 — Modalités de la haute disponibilité PME

Contexte. ADR-001 confie la HA à etcd + Patroni + orchestrateur Go. Sept points restaient à trancher pour la pile PME réelle, répartie sur plusieurs machines. Le plus structurant : etcd n'opère qu'avec une majorité de ses membres ; avec deux membres, la perte de **l'une ou l'autre** machine arrête etcd, et Patroni repasse alors le primaire en lecture seule par sécurité. Une question de fond a aussi été posée : retirer l'élection du nœud Rust fait-il perdre la garantie de quorum ?

Décisions (2026-10-28).

#	Sujet	Décision
1	Clusters à 2 serveurs PostgreSQL	**Témoin etcd sur une 3e machine de la PME** : etcd seul, sans PostgreSQL ni données. La majorité etcd survit à la perte d'une machine ; la bascule automatique devient possible ; tout reste chez la PME
2	Réplication synchrone	**Stricte à partir de 3 serveurs PostgreSQL** (écritures bloquées si aucun standby synchrone) ; **en dessous, non stricte avec alerte visible** dans la console : le cluster continue d'écrire quand son standby tombe, mais la PME est prévenue qu'elle n'est plus protégée
3	Version PostgreSQL	**17** (image Spilo), une seule version à maintenir
4	Clusters déjà installés	**Réinstallation des bancs** (Pilote AMANE, AMANE Test) ; pas de migration de données
5	Accès au primaire	**HAProxy local** sur chaque machine, qui interroge Patroni (`GET /primary`) : adresse fixe pour le nœud et pour l'application externe
6	Chiffrement du trafic entre machines	**Reporté à l'étape 4** (mesh WireGuard, `orchestrator-go/mesh`) ; ce trafic reste dans le périmètre PME
7	Source de l'époque	**Timeline PostgreSQL** : incrémentée à chaque promotion, persistée, identique sur tous les nœuds, ne recule jamais

Le quorum reste une garantie du socle. Ce qui sort du nœud Rust, c'est la **décision** de promouvoir (désormais prise par Raft via etcd, et appliquée par Patroni). Ce qui reste dans le socle et dans l'image du nœud, c'est la **vérification** et l'**application** :

- `quorum.rs` est réorienté en **garde de quorum en lecture** : une fonction pure classe l'état lu dans etcd/Patroni en « sain », « sous le seuil » ou « quorum perdu ». Elle reste testable unitairement ; elle ne promeut plus rien.
- `epoch.rs` et `fencing.rs` sont conservés ; l'époque est la timeline PostgreSQL.
- Le nœud contrôle son époque **en continu** (et non plus au seul démarrage) : époque dépassée → plus d'écriture dans `ss-journal`, plus d'annonce comme primaire ; cluster sous le seuil → signalement au portail.

Limites de l'existant que cette décision corrige. L'élection de `quorum.rs` reposait sur les sondes de chaque nœud (des partitions asymétriques donnent des vues divergentes), ne persistait aucune décision, tirait l'époque d'un compteur local vérifié au seul démarrage, et ne rétrogradait jamais un primaire isolé.

Preuves attendues. Chaque promesse est vérifiée par un scénario de chaos automatisé sur un laboratoire simulant trois machines (sous-étape 2a) : arrêt brutal du primaire, primaire isolé (doit cesser d'écrire), partition minoritaire (aucune promotion), cluster 2 + témoin, perte d'un standby en mode strict et non strict.

4. Tri du dépôt

4.1 À garder intact

Composant	Justification
`spike/crates/ss-crypto` (AK, DEK, CRL, Shamir, device_key, recovery)	Socle de la souveraineté, indépendant de tout métier
`spike/crates/ss-journal`	Journal CBOR chiffré append-only, générique
`ss-consensus/src/epoch.rs`, `fencing.rs`	Anti split-brain générique (lease etcd)
`spike/relay/`, `relay-deploy/`	Le relais zero-knowledge convient tel quel
`proto/amane/framework/v1/`, `tests/contracts/`, `buf.gen.yaml`	Contrat figé : on pourra ajouter des messages, jamais modifier l'existant
`orchestrator-go/` (consensus, supervisor, grpcserver, mesh, replication, telemetry)	Devient le moteur HA de production (ADR-001)
Modèles `Tenant`, `License`, `Device` ; `devices/api_register.py` ; `api/`	Comptes, licences et identité d'installation inchangés
`.github/workflows/`, `.ci/`, `scripts/*_measure.sh`, `docs/bench/`	Garde-fous CI et mesures de bascule

4.2 À garder mais adapter

Composant	Adaptation prévue
`spike/node`	Devient l'**hôte d'exécution** : garde init, run, status, delist, le client relais et le heartbeat SaaS ; gagne le lancement et la supervision de l'application externe à partir de son manifeste, la base PostgreSQL gérée, et un reverse proxy qui sert l'interface de l'application telle quelle
`core-rust/ss-client`	N'est plus un SDK intégré par l'application (elle reste inchangée). `license.rs`, `pairing.rs` et `offline.rs` sont rapatriés dans le nœud ou conservés comme bibliothèque interne
Django (`tenants/`, `licenses/`, `devices/`, `dashboard/`)	Nouveau modèle **Application** (image soumise, digest, manifeste : ports, volumes, base requise, variables), rattaché à la licence ; génération des installeurs et du compose incluant l'application
`templates/tenants/*.tmpl`, `pme-deploy/`	Ajout du service applicatif et de la pile etcd/Patroni/orchestrateur (ADR-001) ; `watchtower` revu (mise à jour par digest vérifié uniquement) ; `pgadmin` rendu optionnel
`frontend/`	Garde Tenants, Licences, Appareils, Clusters, Relais, MonEspace, Login, Register, Déploiement ; ajoute une page Applications ; met à jour la page Architecture
`ss-consensus/src/supervision.rs`	Réduit à la lecture d'état PostgreSQL (plus de promotion par le nœud)
`ss-consensus/src/quorum.rs`	**Reclassé (ADR-004)** : initialement à supprimer, il est réorienté de l'élection vers une garde de quorum en lecture (sain / sous le seuil / perdu)

Composant	Adaptation prévue
`README.md`, `CLAUDE.md`, `ACCES-INTERFACES.md`, `CONTRAT-API.md`, `Architecture_SaaS_Souverain_Trois_Acteurs.md`	Réécrits une fois la refonte stabilisée

4.3 À supprimer

Élément	Raison
`spike/node/src/stock/` et les routes HTML liées au stock	Métier AMANE remplacé par le SaaS externe
`core-rust/ss-client/src/invariants.rs`	Invariants propres au stock
`frontend/src/pages/StockPage.tsx`	Écran du métier supprimé
`proto/framework.proto`	Vestige V1, même paquet que le contrat qui fait autorité
Console Django en double : `templates/**/*.html`, `views_web.py`, `views_espace.py`, `dashboard/views*.py`, `urls.py` web, `static/`	Doublon de la console React (couverture vérifiée écran par écran). Les `.tmpl` (générateurs d'installeurs) sont conservés. **Correction** : les `views.py` de `tenants/`, `licenses/` et `devices/` sont les ViewSets DRF de l'API et restent en place
`spike/node/Dockerfile.stub`, `.gitkeep` inutiles	Vestiges
`fix_locks.py`, `verify.py`, `docker-cleanup.ps1`	Scripts ponctuels ; `fix_locks.py` contient un mot de passe en clair
`.agenthub/sessions/`	Sans rapport avec le projet
`paroles_explication_mission_b.txt`	Note personnelle
`impact_missions_v1_vers_v2 (1).md`, `Stack_Technique_Framework_Souverainete_SaaS (1).md`, `schema_soutenance.html`	Déplacés vers `docs/archive/` plutôt que supprimés (le schéma de soutenance n'a pas d'équivalent React)

5. Plan de travail par étapes

Étape	Contenu	Statut
0	Branche `main3`, instantané V2, présent document	Terminée
1	Nettoyage : suppressions du § 4.3, un commit par catégorie	Terminée
2a	Image Docker de l'orchestrateur ; laboratoire simulant trois machines ; scénarios de chaos prouvant chaque promesse (ADR-001, ADR-004)	Terminée
2b	Gabarit compose PME (etcd, Patroni, orchestrateur, HAProxy), installeurs, génération par le portail, tests Django	À faire
2c	Recentrage du nœud Rust : retrait de la promotion, garde de quorum en lecture, époque = timeline, fencing continu, séparation de `NODE_MODE`	À faire
2d	Réplication synchrone et alerte sous le seuil, mesures de bascule sur le banc réinstallé	À faire
3	Nœud → hôte d'exécution : manifeste, lancement et supervision d'une image Docker, reverse proxy de l'interface (ADR-002)	À faire
4	Confinement et chiffrement : réseau interne sans sortie, données de l'application chiffrées au repos, sauvegardes chiffrées via le relais (ADR-003)	À faire
5	SaaS éditeur : modèle Application, soumission d'image par digest, génération des installeurs	À faire
6	Console React : page Applications, mise à jour Architecture	À faire
7	Documentation : README, CLAUDE.md, accès, contrat d'API	À faire
Plus tard	Binaires Linux et .exe Windows (ADR-002)	Prévu

6. Journal des étapes

Étape 0 — Préparation (2026-10-28)

- Création de la branche `main3` depuis `main2` (`e0f6d09`) et publication sur `origin/main3`.
- Commit `6da5022` « chore: état V2 avant nettoyage » : 76 fichiers V2 jusque-là non suivis ou non commités (pages React clusters, relais, espace tenant ; API DRF clusters, relais, statistiques, images ; quorum `ss-consensus` ; documents d'accès et de déploiement).
- Inventaire du dépôt et tri initial (§ 4).
- Décisions ADR-001, ADR-002 et ADR-003 actées.
- Création du présent document et de son générateur PDF.

Étape 1 — Nettoyage (2026-10-28)

Quatre commits, un par catégorie, pour que chacun reste lisible et réversible.

Commit	Contenu
`7ca8e78`	Racine : retrait de `fix_locks.py` (mot de passe en clair), `verify.py`, `docker-cleanup.ps1`, `.agenthub/`, d'une note personnelle ; deux notes de cadrage archivées dans `docs/archive/`
`81d7081`	Vestiges : `proto/framework.proto` (V1), `spike/node/Dockerfile.stub`, `.gitkeep` inutiles, dossier `infra/` vide. `CLAUDE.md` et le serveur gRPC pointent vers le seul proto restant
`5609fb6`	Métier stock : module `spike/node/src/stock/`, `ss-client/src/invariants.rs`, `StockPage.tsx` (routée nulle part). La racine `/` du nœud affiche une page « Application hébergée », futur point d'entrée du reverse proxy (étape 3). Authentification et console cluster du nœud conservées
`dc06197`	Console Django : vues et gabarits HTML retirés ; les fonctions partagées avec l'API passent dans des modules `services.py` (historique conservé), la voie PME par jeton dans `tenants/api_pme.py`. Aucune URL d'API ne change

Choix faits pendant l'étape.

- `quorum.rs` reste en place jusqu'à l'étape 2 : le retirer avant qu'etcd/Patroni n'arrivent dans la pile PME supprimerait toute bascule automatique chez les clients.
- Les tables `articles` et `mouvements_stock` déjà créées dans les bases PME ne sont pas supprimées : le nœud ne les crée plus, mais effacer des données client n'est pas du ressort d'un nettoyage de code.
- Le schéma de soutenance, sans équivalent React, est archivé plutôt que supprimé.

Vérifications.

- Django : `manage.py check` sans erreur ; 75 tests exécutés sur SQLite en mémoire, 74 réussis. L'échec (`test_options_de_replication_postgres`) existait déjà avant le nettoyage (vérifié sur le commit `098f1de`), voir § 7.
- Frontend : `npm run build` et `npm run lint` sans erreur.
- Go : `go build ./...` sans erreur.
- Rust : **non compilé** pendant l'étape (Docker arrêté sur le poste, aucun toolchain local) ; diff relu à la main. Compilation confirmée ensuite, voir les compléments ci-dessous.

Étape 1 — Compléments (2026-10-28)

Compilation Rust (via Docker, `rust:1.88-bookworm`).

- `spike/` : compile, 74 tests réussis, 0 échec.
- `core-rust/ss-client` : compile, 7 tests unitaires réussis.
- Les avertissements restants (`delete_session` jamais appelée, champ `node_id` jamais lu, variable `e` inutilisée dans `ss-journal`) existaient avant le nettoyage ; ceux du module stock ont disparu avec lui.

Test d'intégration `core-rust/ss-client/tests/mock_server.rs` retiré. Il implémentait le contrat V1 (`RegisterNode`, `WriteOperation`, `GetClusterStatus`) alors que `build.rs` génère les bindings depuis le contrat V2 : 25 erreurs de compilation, déjà présentes avant le nettoyage (vérifié sur `098f1de`). Il empêchait `cargo test` de lancer les tests unitaires de `ss-client`. Une version alignée sur le contrat V2 pourra être écrite si le besoin revient.

Test `test_options_de_replication_postgres` corrigé. Le test attendait `max_wal_senders=3` et `wal_keep_size=64` ; le gabarit compose (comme `pme-deploy/`) utilise 5 et 512. Le second écart était masqué par l'arrêt du test au premier échec. Les valeurs du gabarit font foi :

- `max_wal_senders=5` : deux standbys en flux et un `pg_basebackup -Xs` (qui occupe deux émetteurs) doivent pouvoir coexister sur un cluster de trois nœuds ; avec 3, le reclonage d'un standby peut être refusé.
- `wal_keep_size=512` (Mo) : un standby redémarré ou brièvement coupé rattrape son retard sans être recloné ; 64 Mo n'offrent pas cette marge.

Le test est aligné et commenté. Suite Django : 75 tests, 75 réussis.

Constat reporté à l'étape 2. En analysant ce test, la réplication s'est révélée asynchrone de fait (`synchronous_commit=remote_write` sans `synchronous_standby_names`). Correction décidée pour l'étape 2, consignée dans ADR-001.

Étape 2a — Image de l'orchestrateur et laboratoire HA (2026-10-28)

Objectif. Avant de modifier la pile installée chez les PME (2b) et le nœud Rust (2c), prouver que l'architecture décidée par ADR-001 et ADR-004 tient ses promesses quand on la casse. La preuve est apportée par un laboratoire qui simule trois machines sur un seul poste, et par des scénarios de chaos automatisés qui rendent un verdict OK / ÉCHEC.

Ce qui a été livré

Fichier	Rôle
`orchestrator-go/Dockerfile`, `.dockerignore`	Image de l'orchestrateur (binaire Go statique, utilisateur non privilégié, 33 Mo). Elle n'existait pas : l'orchestrateur n'était lancé qu'à la main
`ha-lab/docker-compose.yml`	Laboratoire : trois « machines » simulées, deux topologies
`ha-lab/chaos.sh`	Les scénarios de chaos et leur bilan (topologie `temoin`, `trois-pg` ou `tout`, filtre de scénario optionnel)
`ha-lab/README.md`	Notice du laboratoire
`orchestrator-go/supervisor/beacon.go`, `supervisor.go`, `consensus/client.go`	Trois défauts corrigés (voir plus bas)

Fichier	Rôle
`orchestrator-go/supervisor/beacon_integration_test.go`	Test de non-régression du défaut de heartbeat
`.ci/testcontainers/main.go`, `.github/workflows/build-go.yml`, `chaos-tests.yml`	etcd de CI réglé comme en production ; nouveau test ajouté à la campagne de chaos CI

Le laboratoire

Chaque « machine » `mN` est un groupe de conteneurs relié à deux réseaux : `lan`, qui joue le réseau local de la PME, et `mN`, qui joue le bus interne de la machine. Débrancher les conteneurs d'une machine du réseau `lan` simule un câble réseau arraché : ils continuent de se voir entre eux, mais plus les autres machines. Sur `lan`, chaque conteneur a une IP fixe et les services se désignent par IP, comme de vraies machines.

Machine	etcd	PostgreSQL + Patroni	Orchestrateur
m1	10.77.0.11	10.77.0.12	10.77.0.13
m2	10.77.0.21	10.77.0.22	10.77.0.23
m3	10.77.0.31	10.77.0.32 (topologie trois-pg)	10.77.0.33 (topologie trois-pg)

Deux topologies, qui correspondent aux deux cas d'ADR-004 :

- **Topologie « témoin »** : deux serveurs PostgreSQL (m1, m2) et un témoin etcd seul sur m3 (décision 1). Réplication synchrone **non stricte** (décision 2).
- **Topologie « trois-pg »** : trois serveurs PostgreSQL. Réplication synchrone **stricte**.

Réglages Patroni communs : `ttl` 20 s, `loop_wait` 2 s, `retry_timeout` 5 s, `synchronous_mode` activé. Chaque topologie est recréée à neuf avant ses scénarios.

Comment les pannes sont simulées

Panne réelle	Simulation dans le laboratoire
PostgreSQL ou Patroni plante, la machine reste en vie	`docker kill -s KILL` du seul conteneur PostgreSQL ; l'etcd et l'orchestrateur de la machine continuent de tourner
La machine s'éteint brutalement (coupure de courant)	`docker kill -s KILL` de tous les conteneurs de la machine
Câble réseau arraché, switch défaillant	Débranchement de tous les conteneurs de la machine du réseau `lan` ; ils restent vivants

Comment le verdict est rendu

- **Écriture acquittée** : un `INSERT` est acquitté si PostgreSQL confirme le `COMMIT` en moins de 5 s. En réplication synchrone, ce `COMMIT` n'est confirmé qu'après réception par le standby synchrone : une écriture acquittée existe donc sur deux machines.
- **Leader** : il est lu dans etcd **auprès de la majorité**. Un membre etcd isolé refuse de répondre (lecture linéarisable), on interroge alors le suivant.
- **Rétrogradation** : on vérifie que PostgreSQL n'accepte plus d'écriture (`pg_is_in_recovery()` vrai, ou PostgreSQL arrêté). Le rôle annoncé par l'API REST de Patroni ne suffit pas : un primaire privé d'etcd peut encore renvoyer son dernier rôle connu alors que Patroni a déjà arrêté PostgreSQL.

- **Zéro perte** : un marqueur est écrit et acquitté juste avant la panne ; il doit se retrouver sur le nouveau primaire.
- **Retour à la normale** : après chaque scénario, le cluster doit se reformer complètement (tous les membres présents, un leader, au moins un standby synchrone).

Les scénarios et les promesses qu'ils prouvent

1. Crash du PostgreSQL primaire. On tue le seul processus PostgreSQL du primaire. On vérifie qu'un autre nœud devient primaire et acquitte des écritures, que le marqueur est présent (zéro perte), puis qu'au redémarrage l'ancien primaire revient **en réplica** et non en second primaire. *Promesse : bascule automatique sans perte, et fencing de l'ancien primaire.*

2. Panne de la machine du primaire. On tue toute la machine du primaire : son etcd, son PostgreSQL, son orchestrateur. On vérifie la bascule et le zéro perte. Dans la topologie témoin, c'est le témoin etcd qui fournit la majorité : sans lui, etcd s'arrêterait et Patroni bloquerait toute écriture. *Promesse : la majorité restante suffit à basculer (décision 1 d'ADR-004).*

3. Isolement réseau du primaire. C'est le scénario le plus exigeant : le primaire est coupé du réseau mais reste vivant. Pendant qu'une écriture est tentée sur ce primaire isolé, on surveille la bascule côté majorité. On vérifie que **le primaire isolé n'acquitte aucune écriture**, que la majorité élit un nouveau primaire, que l'ancien cesse lui-même d'accepter des écritures, et qu'au retour du réseau **l'écriture non acquittée n'apparaît nulle part** dans le cluster (elle est effacée par `pg_rewind`). *Promesse : jamais deux primaires capables d'acquitter une écriture (anti split-brain).*

4. Perte du standby, non strict (topologie témoin). On tue la machine du standby. On vérifie que les écritures reprennent rapidement sur le primaire, désormais sans standby synchrone. *Promesse : en dessous de 3 serveurs PostgreSQL, le cluster continue d'écrire (décision 2) ; l'alerte « vous n'êtes plus protégé » est à ajouter à l'étape 2d.*

5. Isolement du témoin etcd (topologie témoin). On coupe le témoin du réseau pendant 15 s. On vérifie que le primaire ne change pas et que les écritures continuent. *Promesse : le témoin ne crée pas de point de défaillance.*

6. Partition minoritaire d'un standby (topologie trois-pg). On isole un standby pendant 30 s. On vérifie qu'il ne se promeut jamais, que le primaire reste le même et qu'il continue d'acquitter des écritures grâce à l'autre standby. *Promesse : un nœud minoritaire ne peut jamais devenir primaire.*

7. Blocage strict sans standby (topologie trois-pg). On tue les deux standbys. On vérifie que le primaire **n'acquitte plus aucune écriture**, puis que les écritures reprennent au retour des standbys. *Promesse : en mode strict, aucune écriture n'est confirmée si elle n'existe pas sur deux machines (décision 2).*

Les scénarios 1 à 3 sont joués sur les deux topologies : 10 exécutions au total.

Résultats (campagne finale)

Scénario	Témoin (2 PG + témoin)	Trois-pg (3 PG, strict)
1. Crash du PostgreSQL primaire	OK — bascule en 3,7 s, zéro perte	OK — bascule en 3,3 s, zéro perte
2. Panne de la machine du primaire	OK — bascule en 5,7 s, zéro perte	OK — bascule en 4,8 s, zéro perte
3. Isolement réseau du primaire	OK — aucune écriture acquittée par l'isolé ; bascule en 6,3 s ; l'isolé n'accepte plus d'écriture après 9,5 s ; écriture non acquittée écartée	OK — mêmes vérifications ; bascule en 7,3 s ; isolé hors service après 9,8 s

Scénario	Témoin (2 PG + témoin)	Trois-pg (3 PG, strict)
4. Perte du standby, non strict	OK — écritures reprises en 1,2 s	—
5. Isolement du témoin	OK — aucun effet pendant 15 s	—
6. Partition minoritaire	—	OK — aucune promotion en 30 s
7. Blocage strict	—	OK — aucune écriture acquittée sans standby, reprise au retour

Toutes les promesses de sécurité sont tenues : zéro perte, aucun double primaire, aucune promotion minoritaire, blocage strict effectif.

Défauts trouvés et corrigés dans l'orchestrateur

Les premières campagnes ont mis au jour trois défauts réels du superviseur Go, invisibles dans les tests existants qui ne simulaient pas de partition réseau entre machines.

Défaut	Effet constaté	Correction
Client etcd sans keepalive gRPC	Après une partition, le client restait accroché à sa connexion vers le membre etcd isolé : heartbeats, droit de forcer et libération du verrou échouaient tous. La bascule n'était plus accélérée et retombait sur le `ttl` de Patroni (21 à 26 s)	Keepalive d'une seconde (`consensus/client.go`). Le serveur etcd doit l'accepter : `--grpc-keepalive-min-time 1s` (laboratoire, CI, et pile PME à l'étape 2b)
Délai de libération du verrou Patroni trop court	La suppression du verrou dans etcd avait 300 ms (le délai d'une sonde REST) ; juste après une partition, elle échouait et toute la détection repartait de zéro	Délai d'écriture etcd dédié de 2 s (`supervisor.go`)
Heartbeat jamais rétabli après une isolation (défaut de sécurité)	Une fois reconnecté, l'orchestrateur réutilisait une lease etcd expirée et ne battait plus jamais le cœur de son nœud. Si ce nœud redevenait primaire, la garde anti-partition ne le protégeait plus : une simple coupure REST aurait pu déclencher une bascule forcée à tort	Session recréée dès que la lease est morte (`beacon.go`). Test d'intégration `TestHeartbeatSessionRecreatedAfterLeaseLoss` : **il échoue sur le code d'origine et passe avec la correction**

Effet mesuré des corrections sur l'isolement du primaire : de 21,8 à 25,8 s avant, à 6,3 et 7,3 s après.

Artefacts de laboratoire corrigés

Ces écarts tenaient au laboratoire ou au script de test, pas à l'architecture ; ils sont consignés parce que certains renseignent l'étape 2b.

- **Adresse annoncée par Spilo.** Spilo ignore `PATRONI_POSTGRESQL_CONNECT_ADDRESS` et lit `PG_CONNECT_ADDRESS` / `RESTAPI_CONNECT_ADDRESS`, avec pour défaut l'IP du conteneur. Après une reconnexion réseau, l'adresse annoncée était périmée. *Pour 2b : annoncer l'IP LAN de la machine.* Le compose de développement est concerné.
- **DNS d'un conteneur arrêté.** La résolution du nom d'un conteneur arrêté met environ 3 s à échouer, ce qui retardait la promotion. D'où les IP fixes, plus fidèles à une PME.
- **Rôle REST en cache.** Voir « Rétrogradation » ci-dessus.

- **Rythme des scénarios.** Voir le cooldown ci-dessous.
- **Mesure de l'isolement.** La tentative d'écriture sur le primaire isolé est menée en parallèle de la surveillance de la bascule ; menée avant, elle ajoutait 5 s à la mesure.

Comportements à connaître

- **Cooldown de 30 s.** Après une bascule forcée, l'orchestrateur s'interdit d'en forcer une autre pendant 30 s (anti-oscillation). Une seconde panne dans cette fenêtre n'est rattrapée que par le `ttl` de Patroni (environ 20 s).
- **Rétrogradation hors ligne.** Un primaire privé d'etcd est **arrêté** par Patroni (« demoting self, offline ») et reste arrêté tant que le réseau n'est pas revenu ; il rejoint ensuite le cluster en réplica.
- **Passage en asynchrone (non strict).** Quand le standby d'un cluster à 2 serveurs tombe, Patroni retire le standby synchrone et les écritures reprennent en asynchrone. C'est le comportement voulu par la décision 2 ; l'alerte reste à construire (2d).
- **Option `synchronous_mode_stop_on_limit`.** Vérifié dans le code de Patroni 4.0.4 (image Spilo) : cette option n'existe pas et est ignorée. Seules `synchronous_mode` et `synchronous_mode_strict` existent. Le compose de développement est à corriger.

Délais de bascule et cible de 5 s

Les crashs de processus basculent en 3,3 à 3,7 s et la panne d'une machine en 4,8 à 5,7 s, autour de la cible. L'isolement réseau prend 6,3 à 7,3 s. Ce délai a une borne basse structurelle : avant de forcer, l'orchestrateur doit constater que le heartbeat du primaire a expiré (lease de 2 s, puis 2 confirmations à 0,5 s d'intervalle). Ce délai est justement ce qui empêche de forcer une bascule sur une simple coupure REST. Ensuite, Patroni ne traite la demande qu'à son tour de boucle suivant (`loop_wait` 2 s). Deux leviers sont à évaluer à l'étape 2d, sur le banc réel : `loop_wait` à 1 s, et la durée du heartbeat. Les deux augmentent le risque de fausse détection ; la cible sera tranchée sur mesures réelles.

Vérifications

- Go : `go build`, `go vet`, tests unitaires de tous les paquets réussis.
- Intégration sur etcd réel (harnais CI) : `consensus`, `supervisor` et `mesh` réussis, dont le nouveau test de non-régression ; ce dernier échoue sur le code d'origine.
- Chaos : 10 scénarios sur 10 réussis sur la campagne finale.
- Aucun changement Django, React ni Rust dans cette sous-étape.

7. Points ouverts

Sujet	Question	Lié à
Chiffrement au repos de PostgreSQL	Quel mécanisme : volume chiffré (LUKS, gocryptfs) avec clé dérivée de la DEK, ou chiffrement applicatif ? PostgreSQL n'offre pas de TDE natif	ADR-003
Licence du SaaS externe	Comment l'application vérifie-t-elle sa propre licence sans accès sortant vers son éditeur ? Relais de métadonnées par le nœud ?	ADR-003
Mises à jour de l'application	Signature des images (cosign) en plus du digest ? Qui valide une nouvelle version ?	ADR-002
Authentification utilisateur	Le reverse proxy ajoute-t-il un contrôle d'appareil appairé devant l'interface, ou laisse-t-on uniquement le login de l'application ?	ADR-003
Applications multi-conteneurs	Un SaaS peut nécessiter plusieurs services (web, worker, cache) : le manifeste doit-il décrire un ensemble ?	ADR-002
Bases autres que PostgreSQL	Que faire d'une application qui exige MySQL ou MongoDB ?	ADR-001
Formation du cluster etcd	Comment une machine qui rejoint un cluster existant devient-elle membre etcd (liste générée par le portail, ajout de membre) sans intervention manuelle ?	ADR-004, étape 2b
Délais de bascule	Isolement réseau à 6–7 s, panne de machine autour de 5 s : régler `loop_wait` et la durée du heartbeat sur le banc réel, sans augmenter les fausses détections	ADR-004, étape 2d
Adresses annoncées par Patroni	Dans la pile PME, `PG_CONNECT_ADDRESS` et `RESTAPI_CONNECT_ADDRESS` doivent valoir l'IP LAN de la machine (ports publiés), etcd doit accepter le keepalive d'une seconde	Étape 2b
Compose de développement	Retirer `synchronous_mode_stop_on_limit` (inexistante) et remplacer les `PATRONI_*_CONNECT_ADDRESS` ignorées par Spilo	Étape 2b
Filtres de chemins de `chaos-tests.yml`	Le workflow surveille `consensus/**`, `supervisor/**`… à la racine, alors que ces dossiers sont sous `orchestrator-go/` : il ne se déclenche jamais sur une PR	CI

Annexe — Mettre à jour ce document

1. Modifier la source `docs/refonte-v3/AMANE-Refonte-V3.md` : statut des étapes (§ 5), entrée dans le journal (§ 6), nouvelle ADR (§ 3) ou point ouvert (§ 7).
2. Incrémenter la version dans l'en-tête et ajouter une ligne à l'historique des versions.
3. Régénérer le PDF : `python docs/refonte-v3/generer_pdf.py` (dépendance : `reportlab`).
4. Commiter la source et le PDF ensemble, par exemple `docs(refonte): étape 1 — nettoyage`.